
OPERATOR AND TECHNICAL LEAD MANUAL

agenttalk new-user manual

Architecture, messaging, state flow, dashboard operation, skills, supervised agents, evidence, and recovery.

Release baseline: v0.74.0

Published: 2026-07-12

Project: agenttalk

Contents

1. Read this first

- Mental model

2. Core invariants

3. Install and initialize

4. Identity, roster, roles, and liaison

5. Messaging and thread states

- Message kinds

- Thread state machine

6. First workflow: two agents

7. Skills

8. Team workflows

- Lead-led team

- Escalation and operator answers

- Broadcast

- Proposals and consults

9. Dashboard

- If you can type an answer

10. Supervisor, wrapper, and liveness

11. Lead-loop controller

12. Domains, lanes, and worktrees

12a. Onboarding an existing codebase

13. Gates, close records, and assurance

14. Knowledge and lessons

15. Recovery and troubleshooting

16. State reference

- Thread states

- Thread next-action hints

- Gate verdicts

- Gate statuses

- Close lifecycle

- Lane delivery lifecycle

- Lesson statuses

- Dead-letter states

- Supervisor state examples

17. Command map

18. Glossary

Overview

Audience: new operators and technical leads adopting agenttalk for a local coding-agent pair or team.

Goal: understand the system first, then operate it safely: messaging, thread states, dashboard views, skills, workflows, supervision, lanes, knowledge, gates, close records, and recovery.

Last updated: 2026-07-12. Current release baseline: v0.74.0.

agenttalk is a local, file-backed coordination platform for coding-agent CLIs such as Claude Code and Codex. It lets separate agent windows talk directly, track who owes the next move, route human decisions through one liaison, and record evidence for reviews and releases.

agenttalk is not a sandbox, a malicious-peer defense, an enterprise auth system, or an autonomous project manager. It makes disciplined teamwork visible and auditable. Git, the OS, CI, and the human operator remain the real authority boundaries.

1. Read this first

The fastest way to understand agenttalk is to separate five layers:

Layer	What it answers	Main files and commands
Store	What happened on the bus?	.agenttalk/messages/*.json, send, reply, recv, drain, wait
Thread projector	Who owes the next move?	threads, sync, dashboard Active threads
Team control	Who is on the team and who talks to the operator?	roster, roles, groups, operator_facing, escalate, relay
Work and evidence	Is this scoped, reviewed, tested, and safe to close?	domain, lane, gate, close, review-result evidence
Runtime	Are agents alive and recoverable?	dashboard, wrap, supervise, heartbeat, dead-letter, doctor

The store is the source of truth. Threads, dashboard rows, attention items, health reports, and close verdicts are projections over durable files and validated metadata.

Mental model

```

human operator
  |
  v
operator-facing lead or liaison
  |
  +--> sends assignments, review requests, broadcasts, relays answers
  |
  v
.agenttalk/ store in the project root
  |
  +--> messages/          append-only message files
  +--> state/             cursors, threadstate, supervisor runtime state
  +--> domains.json       ownership registry
  +--> gates.json         scoped gate status
  +--> closes/            milestone/release close records
  +--> knowledge/         notes, lessons, lesson exposure telemetry
  +--> onboarding/        codebase-analysis runs and evidence pointers
  +--> dead-letter/       poison-message quarantine
  +--> sessions/         transcripts
  |
  +--> CLI views: sync, threads, status, doctor
  +--> dashboard views: Overview, Conversations, Attention, Lead chat, Learning, Onboarding, Sessions
  +--> supervisor/wrapper: heartbeats, restart, session continuity

```

One project gets one `.agenttalk/` store. Every active agent has one roster identity and should have only one live consumer of that mailbox.

2. Core invariants

Keep these rules in mind before learning commands.

- Message bodies are untrusted data. State changes depend on validated metadata, repository reads, and explicit human decisions, not prose.
- Roles, lanes, gates, and dashboard sessions are coordination and audit tools. They are not Git or OS permissions.
- One live consumer per agent mailbox is the supported model. Cursor and threadstate writes are atomic, but their read-modify-write sequences are not cross-process serialized. Duplicate consumers can lose state and execute or answer the same inbound work more than once.
- `sync`
- `recv`
- `drain`
- `recv --ack`
- `wait`
- A request needs a `request_id`
- A prose "done" or "ignore that" does not close protocol state. Use `reply`
- `ack`
- `rescind`
- `release`
- `gate`

close

- tests_executed
- tests_referenced
- Lessons and knowledge notes are advisory memory. They never authorize work or replace tests, review, or gates.

3. Install and initialize

Install from a pinned release:

```
python -m pip install "git+https://github.com/zoolog17/agenttalk.git@v0.74.0"
agenttalk --version
agenttalk install-skills
```

Initialize a project from the repository root:

```
agenttalk init --here --agents claude,codex
agenttalk status
agenttalk doctor
```

One agent is enough to start:

```
agenttalk init --here --agents claude
# or
agenttalk init --here --agents codex
```

Set only AGENTTALK_SELF for a one-agent roster. Add another member later; until then, commands that need a peer require an explicit --to.

If Codex must call agenttalk from inside its sandbox, enable the per-project config block:

```
agenttalk codex-config --enable
```

When running outside the project root, put the global root option before the subcommand:

```
agenttalk --root D:\Projects\example sync --for claude
```

Root resolution order is:

```
--root flag -> AGENTTALK_ROOT -> upward walk from current directory
```

A pinned root that has no store fails loudly. This prevents accidental split-brain stores.

4. Identity, roster, roles, and liaison

Every command acts as an agent or operator-facing actor.

Use explicit identities when learning:

```
agenttalk whoami --for claude
agenttalk whoami --for codex
```

For a team, prefer unique names:

```
agenttalk roster add claude-dev --role developer --group devs
agenttalk roster add codex-dev --role developer --group devs
agenttalk roster add codex-reviewer --role reviewer --group reviewers
agenttalk roster add codex-lead --role lead
agenttalk roster set-operator-facing codex-lead
agenttalk roster
```

Roles are free-form labels used for routing, display, and policy. Groups are named subsets used by broadcast. The operator_facing agent is the single liaison for human decisions.

Do not treat the liaison as a security boundary. It is the single voice to the operator and an audit identity.

5. Messaging and thread states

Message kinds

The bus accepts a fixed vocabulary of message kinds. Important kinds are:

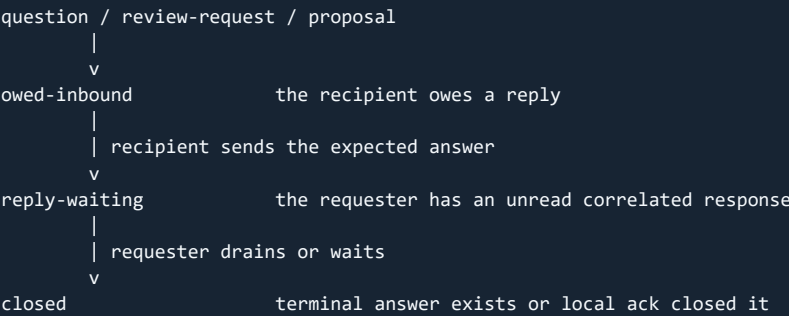
Kind	Purpose
message, note	Generic communication or FYI
question	Opens a tracked request; any non-control answer from the recipient closes it
review-request	Opens a review thread; expects review-result
review-result	Review verdict; use meta.status=approved, rejected, or needs-info
proposal	Concrete proposal; expects proposal-response
proposal-response	Proposal verdict; accepted, rejected, or countered
broadcast command output	Fan-out copies, each with the same broadcast_id and request_id
wake	Low-latency state-change signal, commonly for spec-kitty loops
composing	Control-plane hint that a reply is in progress
rescind	Supersedes the requester's own tracked request
release	Stand-down signal requiring --relay-human or --emergency plus a reason
end	Export a local transcript and send a session-ended signal; not the same authority envelope as release

Unknown kinds are rejected or skipped so a typo does not become a new instruction surface.

Statuses are exact, case-sensitive enums. approved and rejected terminate a review; needs-info returns the ball to the requester. All three proposal responses terminate their proposal. Missing status remains readable in legacy history but is nonterminal; an invalid present status is rejected or skipped.

Thread state machine

Threads are derived from validated messages. They are not separate task objects.



Special transitions:

```
review-result status=needs-info
  -> ball moves back to requester
requester message/note answer
  -> ball moves back to reviewer

rescind by original requester
  -> closed-superseded for every participant

ack --to-request
  -> local closed view for that agent only
```

Thread states from one agent's perspective:

State	Meaning	Typical next action
owed-inbound	The ball is on you	Reply or escalate
reply-waiting	A correlated response is unread in your inbox	Drain, wait, or inspect
open-outbound	You are waiting on someone else	Wait, nudge, rescind, or re-plan
closed	Terminal response or local closure exists	No action
closed-superseded	The requester rescinded it	Stop acting on it; re-ask with a fresh id

Inspect obligations:

```
agenttalk threads --for codex-lead
agenttalk threads --for codex-lead --all
agenttalk sync --for codex-lead
```

Before irreversible work tied to a request:

```
agenttalk check --for codex-dev --to-request <request-id>
```

Exit 0 means current. Exit 3 means superseded. Exit 4 means unknown.

6. First workflow: two agents

1 Start one terminal per agent in the project root.

2 Confirm identity:

```
agenttalk whoami --for claude
agenttalk whoami --for codex
```

3 Send a tracked question:

```
agenttalk send --from claude --to codex --kind question --subject first-check --meta
  request_id=q-first-check -m "Can you see this?"
```

4 Consume it:

```
agenttalk drain --for codex
```

5 Reply on the same request:

```
agenttalk reply --from codex --to-request q-first-check -m "Yes."
```

6 Rejoin after any restart:

```
agenttalk sync --for claude
agenttalk threads --for claude
agenttalk status
```

In normal agent operation, agents use the installed skills instead of typing raw commands for every exchange.

7. Skills

`agenttalk install-skills` installs two families.

Bus skills:

Claude Code	Codex	Use
<code>agenttalk.send</code>	<code>agenttalk-send</code>	Send a one-off message
<code>agenttalk.listen</code>	<code>agenttalk-listen</code>	Keep a mailbox armed
<code>agenttalk.handoff</code>	<code>agenttalk-handoff</code>	Send work for review and wait

Claude Code	Codex	Use
agenttalk.consult	agenttalk-consult	Ask a peer before answering
agenttalk.propose	agenttalk-propose	Ask for accept/reject/counter on a plan
agenttalk.lead	agenttalk-lead	Coordinate a named team
agenttalk.sk-loop	agenttalk-sk-loop	Persistent spec-kitty implement/review loop

Dev-discipline skills:

Skill	Purpose
craft-code	Smallest correct production change
test-coverage	Behavior tests for touched behavior
review-code	Diff review for bugs and regressions
write-docs, review-docs, test-docs	Documentation authoring and QA
review-failure-injection, review-contract-drift, review-release-readiness	Specialist review lenses
tester-qa, test-integration, test-security, assurance-scan	Executed evidence and QA
system-review-protocol	Multi-lens release or milestone close

Skills are instructions and workflows. They do not move protocol state by themselves. Typed bus messages, gates, close records, and Git evidence do. Dev-discipline skills should produce typed evidence when they review, test, or approve work; they are not a second role system and do not override gates or close records.

8. Team workflows

Lead-led team

A lead coordinates, tracks replies, and reports status. The lead does not spawn workers and does not override typed state.

Common cadence:

```
agenttalk sync --for codex-lead
agenttalk status
agenttalk threads --for codex-lead
agenttalk send --from codex-lead --to codex-dev --kind question --subject bugfix -m "Build the narrow
  fix and report SHA plus tests."
agenttalk broadcast --from codex-lead --to-group reviewers --kind question --subject review-availability
  -m "Who can review the final SHA?"
```

Escalation and operator answers

Workers should not ask the human in their own terminal when a liaison exists. They should escalate:

```
agenttalk escalate --from codex-dev --decision "Can I delete generated files?" --why "The cleanup would
  remove generated artifacts from the repo." --option yes --option no --recommendation no -m "Need
  operator decision before deleting generated files."
```

The liaison sees it in sync, attention, or the dashboard, then answers with the typed relay path:

```
agenttalk relay operator-answer --to-request esc-... -m "No. Leave them."
```

Broadcast

Broadcast freezes the audience at send time:

```
agenttalk broadcast --from codex-lead --to-group reviewers --kind question --subject api-name -m
  "Approve or object to this API name."
```

Recipients reply to the shared request id:

```
agenttalk reply --from codex-reviewer --to-request b-... -m "approve"
```

If broadcast exits 5, some copies were written and some were missed. Use `broadcast --resume <broadcast-id>` or rescind the batch.

Proposals and consults

Use a proposal when the peer should accept, reject, or counter a concrete plan. Use a consult when you need critique before answering the operator. Neither is a back door for hidden split work. Outside a spec-kitty mission, the human must approve implementation ownership splits.

9. Dashboard

Start the local dashboard:

```
agenttalk dashboard
agenttalk dashboard --port 0
agenttalk dashboard --enable-actions
```

The dashboard is loopback-only. It is for the local operator, not a remote multi-user web app. Actions are disabled unless `--enable-actions` is passed.

The top bar always shows selected-project and path context. If CSS ellipsizes the path visually, its complete value remains available through the element text, title, and accessibility label. In a multi-root dashboard, a stable path-derived `project_id` routes selected-root reads and actions; the label is never write authority. Responses include `root_info {project_id,label,path}`. GET may omit the selector to select `root[0]` or use a unique display label as a legacy best-effort selector. Blank, repeated, unknown, or ambiguous GET selectors return HTTP 400 `bad_root`.

For POST `/api/intent` and `/api/lead-chat`, a multi-root server requires exactly one explicit full `?root=<project_id>`. Labels are forbidden; omission is valid only for a single-root server. Unknown, blank, repeated, ambiguous, or non-full selectors return HTTP 400 `bad_root` before mutation.

Changing the selector to a different project pushes its id into browser history. Back and Forward restore the selected project and refetch its root-bound feeds. Any actual project change clears root-bound drill-ins, caches, the action session, queued-answer text, and generic and lead-chat composer drafts. The client ignores late responses whose project id belongs to the old selection. This protects routing correctness, not access control: every exposed root remains readable to local processes that can reach the loopback server.

Main views:

View	What it shows	Body text?
Overview	Team layout, health, tasks, recent envelope activity	No
Conversations	Traffic graph and active thread list	No
Attention	Human-needed queue: escalations, holds, stuck agents, dead letters	Bounded escalation excerpt only when action replies are enabled
Lead chat	Operator-to-lead direct channel and pending lead decisions	Yes for the direct transcript; pending decision cards may be summaries
Learning	Accepted lessons, curation provenance, and wrapper exposure telemetry	Yes for curated lesson text; no raw bus bodies or prompt blocks
Onboarding	Codebase-analysis runs, segments, claims, drift, and blocking unknowns	No raw bus bodies or prompt blocks; bounded summaries and refs only
Sessions	Full transcript for a selected thread	Yes
Agent detail	Health, supervisor, capacity, recent envelope activity	No

The body-text split is deliberate:

- /api/state
- /api/attention
- /api/learning
- /api/onboarding
- /api/thread/<request-id>
- /api/lead-chat

Use **Learning** when you need to see what the team has captured as durable process memory, how it was justified, and who handled it. The default list is accepted active lessons only. Each row shows the lesson text, trigger, publisher, curator, owner, evidence reference, anchor metadata, exposure count, and recent "surfaced to prompt" events. Proposed, stale, retired, or superseded lessons are diagnostic rows, not default learned context.

If you can type an answer

The **Attention** card shows the typed decision fields and, when dashboard actions are enabled, a bounded **Question** excerpt from the escalation body above the reply box. Use that card for the immediate answer. Open **Conversations** or **Sessions** when you need the complete thread history.

Ask agents to send typed escalation fields so Attention and Lead chat cards are readable even before the full transcript is opened:

```
agenttalk escalate --from <agent> `
  --decision "Which release path should we take?" `
  --why "CI is still red and the release branch is already cut." `
  --option ship `
  --option hold `
  --recommendation "hold until CI is green" `
  --priority urgent `
  -m "Full context for the operator..."
```

The question excerpt is intentionally bounded; it is context for the action, not a replacement for the full transcript.

10. Supervisor, wrapper, and liveness

Manual listen loops are useful while a human watches the terminal. For unattended work, use the wrapper and supervisor.

Scaffold supervision:

```
agenttalk supervise --init
```

The current scaffold writes supervisor configuration and scripts under `.agenttalk/`, including:

```
supervisor.json
supervisor.ps1
supervisor-task.ps1
deadman.ps1
bin/agenttalk.cmd
```

Run the monitor in a dedicated terminal:

```
.\.agenttalk\supervisor.ps1
```

Recommended wrapped launch shape:

```
agenttalk wrap --for codex-dev --cli codex --loop -- codex -a never -s workspace-write -C
D:\Projects\example
```

Liveness flow:

```
wrapper idle wait or child progress
  -> heartbeat is fresh
  -> supervisor reports healthy/idle or working

heartbeat stale and agent is instrumented
  -> supervisor plans stuck recovery or manual restart
```

```
-> old process tree is killed best-effort
-> launch barrier refuses duplicates if a same-agent wrapper survived
-> wrapper reloads session state and re-enters the loop
```

```
valid message repeatedly poisons the wrapper
-> attempt ledger reaches cap
-> payload moves to dead-letter
-> cursor advances past poison
-> operator can inspect, requeue, or resolve
```

Persistence and teardown are fail-closed:

- supervisor-state.json
.bak
- A heartbeat farther in the future than the configured skew allowance cannot authorize freshness.
- Each wrapper waiting marker has a unique generation token. An old wrapper's finally
- On Windows, the turn watchdog uses
os.kill(pid, SIGTERM)
taskkill.exe

Protected agents are the operator-facing liaison and active lead-role agents. The supervisor never auto-kills them. A manual restart of a protected agent requires `--force-protected`; if the protected agent still has a fresh heartbeat, the operator must also acknowledge the live protected kill.

11. Lead-loop controller

The managed lead-loop is an advanced supervised controller for the team mailbox. It splits the human-facing liaison from the headless mailbox owner:

```
operator-facing lead      manual human contact
managed <name>-lead-loop wrapped supervised mailbox owner
```

The controller acquires a renewable lease before consuming the mailbox, renews it while alive, runs proactive cadence ticks on a quiet bus, and stops if it loses ownership. The lease token never goes to the model child.

Use it when the team needs durable, unattended coordination. Do not run a second consumer for the same mailbox.

12. Domains, lanes, and worktrees

Domains describe ownership:

```
agenttalk domain validate
agenttalk domain list
agenttalk domain show docs
```

A lane is a scoped assignment tied to a domain, base SHA, target ref, and path set. By default, `lane assign` provisions an isolated Git worktree.

```
agenttalk lane assign --id docs-manual --from codex-lead --assignee codex-dev --domain docs --base main
--target main --path docs
agenttalk lane workspace --id docs-manual
agenttalk lane check --id docs-manual
agenttalk lane deliver --id docs-manual --from codex-dev
```

Release-class lanes require the provisioned worktree. `--no-worktree` is only valid with `--advisory` plus a recorded reason; that audit record does not prove isolation and cannot satisfy a release close.

Lane delivery is best understood as:

```
assigned/active -> checked -> prepared (hidden/non-consumable)
                  -> publish_pending (generation + instance bound)
                  -> committed delivery evidence
                  -> cleanup_pending|cleanup_failed -> cleanup complete
|
+--> HOLD until scope, registry, epoch, merge, shared-path,
      active-overlap, and gate problems are fixed
```

Retry the same lane deliver after a publication or teardown failure. It resumes the bound transaction, and a committed artifact remains valid while cleanup is pending. Lane verdicts are advisory HOLD/GO evidence, not file locks.

12a. Onboarding an existing codebase

Before a team edits a large existing project, record the analysis pass:

```
agenttalk onboarding create --id ob-api --from codex-lead --title "API onboarding" --base-ref main
agenttalk onboarding record --id ob-api --from codex-dev --kind segment --key cli --status accepted
  --summary "CLI parser and README command reference mapped." --path src/agenttalk/cli.py --path
  README.md
agenttalk onboarding record --id ob-api --from codex-review --kind drift --key docs.cli.reference
  --status open --segment cli --source docs --confidence medium --summary "README command table may lag
  parser help."
agenttalk onboarding record --id ob-api --from codex-test --kind unknown --key release.owner --status
  open --blocking --summary "Need operator confirmation of the release owner."
agenttalk onboarding show --id ob-api
```

Use the dashboard **Onboarding** view to see the same selected-root ledger: segments inspected, confirmed/conflicted claims, open drift, blocking unknowns, and corrupt-ledger warnings. Treat the ledger as recorded evidence. It does not prove the codebase is fully understood and it does not replace review, tests, or gates.

13. Gates, close records, and assurance

Gates are scoped statuses:

Status	Meaning
green	Passing evidence exists
red	Failing evidence exists
unknown	Required evidence is missing or unreadable
skipped	Not run; blocker gates still HOLD
waived	Operator accepted scoped residual risk until an expiry

Blocker gates clear only on validated automation evidence or an active operator waiver.

```
agenttalk gate set --from codex-lead --scope release:v1 --name tests --status unknown --severity blocker
agenttalk gate check --scope release:v1
agenttalk gate waive --from codex-lead --scope release:v1 --name tests --operator codex-lead --reason
  ci-unavailable --expires 2026-07-14
```

Close records aggregate gates, review lenses, sign-offs, remediation, and a frozen revision:

```
agenttalk close open --id rel-071 --from codex-lead --scope release --revision <sha>
  --non-lane-isolation-not-asserted
agenttalk close check --id rel-071
agenttalk close publish --id rel-071 --from codex-lead --verdict go
```

A GO requires the computed close verdict to be GO. A published close is terminal unless reopened through the supported command path.

Close persistence is compare-and-swap. Every existing-record mutation checks the loaded generation and immutable instance id under a per-close lock. A conflict, missing token, or delete/recreate ABA fails closed; reload and retry. Force-open creates a new instance under lock, and legacy closes are upgraded in that lock. Reopen with a new revision clears the old dirty-worktree artifact.

`close publish --bump-barrier` binds one release barrier to the close id, instance, revision, and generation after recomputing gates, sign-offs, and worktree evidence at the serialization boundary. If sending or stamping fails, retry the exact publish. It resumes the unique validated barrier and never sends a second one; duplicate matches HOLD. The close file and bus are not one ACID transaction, so this explicit recovery path is part of the contract.

Sign-off policy booleans must be real JSON booleans, not strings. Counter ids are unique across the whole close, including different lenses. The shared core risk classes are none, unknown, release, device, accessibility, security, performance, persistence, docs-contract, and quality, plus validated namespaced project extensions.

14. Knowledge and lessons

Knowledge notes are pointer-shaped memory:

```
agenttalk knowledge publish --from codex-dev --domain docs --type gotcha --key docs-help-flags
--anchor-kind path --path docs/USER-MANUAL.md -m "Verify documented flags against --help."
agenttalk knowledge curate verify --from codex-lead --domain docs --key docs-help-flags
agenttalk knowledge pull --domain docs
```

Lessons are knowledge notes for repeatable process learning. Accepted, active lessons can surface in sync when their scope or tags match the work:

```
agenttalk sync --for codex-dev --lesson-tag docs
agenttalk knowledge pull --type lesson
```

For wrapped agents, lesson surfacing is automatic. The wrapper selects matching accepted lessons, injects a bounded "Lessons to check" prompt section, and records pointer-only exposure telemetry in `.agenttalk/knowledge/lesson-exposures.jsonl`. Exposure proves the lesson was matched and surfaced. It does not prove the model read or applied it.

The dashboard **Learning** view and `GET /api/learning` show the same audit chain. By default they show accepted active lessons only, plus recent pointer-only exposure events. Use explicit status filters when you need proposals or stale/retired diagnostics.

15. Recovery and troubleshooting

Start with:

```
agenttalk doctor
agenttalk status
agenttalk sync --for <agent>
agenttalk threads --for <agent> --all
```

Symptom	Likely cause	Safe move
Command sees the wrong team	Wrong root	Use <code>agenttalk --root <project> ...</code> ; check <code>whoami</code>
Agent has unread replies	Cursor not advanced	<code>drain --for <agent> or scoped wait --to-request</code>
You are waiting on a stale request	Request was superseded or unknown	<code>check --for <agent> --to-request <id></code>
Dashboard reply box needs more context	Attention card shows a bounded excerpt, not the whole transcript	Open Conversations or Sessions for the full thread; use typed escalation fields
Lead chat unavailable	Liaison heartbeat stale or missing	Start wait, run wrapped, or install activity hook
Duplicate wrappers/windows	Same mailbox consumed twice	Stop duplicate; use wait <code>--refuse-stacked-wait</code> ; inspect supervisor. Exit 6 can also mean an older scoped wait was superseded by a newer same-thread waiter; it did not consume the reply.

Symptom	Likely cause	Safe move
Lane says publication or cleanup pending	A two-phase delivery stopped at a durable checkpoint	Retry the same lane deliver; do not force-reassign or mint another artifact
Close barrier send/stamp failed	Published GO has a recoverable bound barrier	Retry the exact close publish --bump-barrier; do not send an ad hoc barrier
Supervisor state primary is corrupt	A previous valid generation may exist	Inspect the .bak; if both copies are invalid, keep the fail-closed stop and repair deliberately
Poison message blocks wrapped agent	Repeated deterministic failure	dead-letter list, show, requeue, resolve, or purge --resolved
Invalid message files	Corrupt or forged files	prune --invalid --dry-run, then prune --invalid
Store too large	Old closed history accumulated	compact; remember old closed checks can become unknown
Need to clear active bus state	Fresh session needed	reset; know what survives before running it

Dead-letter recovery:

```
agenttalk dead-letter list
agenttalk dead-letter show --agent <agent> --id <message-id>
agenttalk dead-letter requeue --agent <agent> --id <message-id>
agenttalk dead-letter resolve --agent <agent> --id <message-id> --reason handled --from <liaison>
agenttalk dead-letter purge --resolved --from <liaison>
```

resolve closes matching wrapper dead-letter notice escalations so they do not linger as active work. purge --resolved archives resolved payloads and sidecars under .agenttalk/dead-letter-archive/; archived rows are no longer requeueable by the live dead-letter requeue command unless restored.

16. State reference

Thread states

```
owed-inbound
reply-waiting
open-outbound
closed
closed-superseded
```

Typed response statuses:

```
review-result: approved | rejected | needs-info
  terminal: approved | rejected
proposal-response: accepted | rejected | countered
  terminal: accepted | rejected | countered
```

Thread next-action hints

```
reply
read-reply
await-reply
answer-operator
```

Gate verdicts

```
GO
HOLD
```

Gate statuses

green
red
unknown
skipped
waived

Close lifecycle

```
draft/open(instance_id, generation)
  -> checked generation-bound mutations
  -> acked/countered lenses
  -> check HOLD|GO
  -> published HOLD|GO
  -> optional bound barrier pending/send/stamp recovery
```

Lane delivery lifecycle

active -> prepared (non-consumable) -> publish_pending -> committed
committed -> cleanup_pending | cleanup_failed -> cleanup complete

Lesson statuses

proposed
accepted
retired

Dead-letter states

unresolved
requeued
resolved

Supervisor state examples

HEALTHY_IDLE
MANUAL_RESTART
STUCK_RECOVER
LAUNCH_BLOCKED
CONFIG_BLOCKED

The exact supervisor tokens are planner diagnostics. Treat heartbeat freshness and the generated plan as the operational source.

17. Command map

Area	Commands
Setup	init, install-skills, codex-config, doctor, whoami
Identity	roster, roster add, roster set-role, roster set-group, roster set-operator-facing, roster retire
Messaging	send, reply, broadcast, propose, composing, rescind
Reading	recv, drain, wait, sync, threads, status, tail
Operator routing	escalate, relay, attention
Safety	check, barrier, gate, close
Work scope	domain, lane assign, lane workspace, lane check, lane deliver, lane approve-shared
Onboarding	onboarding create, onboarding record, onboarding show, onboarding state, onboarding list
Memory	knowledge publish, knowledge curate, knowledge pull, knowledge search, knowledge onboard

Area	Commands
Runtime	dashboard, serve, start, wrap, supervise, heartbeat, request-restart, request-launch, managed-lead-loop, deadman
Recovery	dead-letter list/show/requeue/resolve, prune, compact, reset

Use `agenttalk <command> --help` for exact flags.

18. Glossary

Agent name : The roster identity a CLI window acts as.

Broadcast : Fan-out that writes one ordinary message per frozen recipient.

Close : An auditable milestone or release verdict over a frozen revision and evidence.

Cursor : The last globally consumed message id for one agent.

Dead letter : A valid message quarantined after repeated deterministic wrapped-agent failure.

Domain : Ownership registry entry for paths and expertise.

Escalation : A tracked operator-input request routed to the liaison or sole lead.

Gate : A scoped HOLD/GO input such as tests, security, or CI.

Knowledge note : Durable pointer-shaped project memory anchored to a file, SHA, symbol, request, or work package.

Lane : A scoped work assignment and deliver gate, usually with a managed worktree.

Lead : A coordinating role. It routes work but is not an authority boundary.

Lesson : A curated knowledge note for repeatable process learning.

Liaison : The `operator_facing` agent that owns the human operator channel.

Request id : Stable correlation id for a tracked question, review request, proposal, or broadcast.

Thread : Derived request/reply state for one `request_id` from one agent's perspective.

Wrapper : `agenttalk wrap`, which owns the bus loop and runs the real CLI as a per-turn child.

Supervisor : Generated monitor scripts and state that launch, watch, and relaunch agents.

Waiver : Explicit operator acceptance of scoped residual risk for a limited time.